

LATIHAN PENGAMATAN

BAB X — Program Aplikasi Pengolah Vektor

Mata Kuliah	: Praktikum Komputer Aplikasi
Topik	: BAB X — Program Aplikasi Pengolah Vektor
Tugas	: Latihan Pengamatan
Nama	: Alven Tendrawan
NIM	: 71230995

Pendahuluan

Dokumen ini berisi penyelesaian dua butir Latihan Pengamatan dari BAB X tentang Program Aplikasi Pengolah Vektor (aplikasi *Corat-Coret Vektor*). Solusi diberikan dalam bentuk modifikasi kode VB.NET yang dapat langsung disisipkan/diganti pada kode aplikasi yang telah dibuat di modul. Setiap soal disertai penjelasan singkat agar mudah dipahami perubahannya.

Soal 1 — Menyimpan Ukuran Garis Tepi sebagai Data Vektor

Soal: Buatlah agar ukuran garis tepi ikut disimpan sebagai data vektor. Tuliskan kode yang dibutuhkan untuk menyimpan dan membukanya kembali.

Strategi Penyelesaian

Pada kode awal, ukuran garis tepi (`tepi.Width`) tidak ikut ditulis ke `TextBox1`, sehingga saat *Gambar Ulang!* ditekan, semua objek digambar dengan tebal yang sedang aktif — bukan tebal saat objek dibuat. Solusinya: menambahkan satu baris vektor baru bertipe `o tebaltepi` yang ditulis sebelum setiap baris objek (`garis/kotak/elips/dll`). Saat membuka kembali (*Gambar Ulang*), baris ini dideteksi pada `Select Case` lalu nilainya digunakan untuk mengubah `tepi.Width`.

a) Modifikasi pada event `MouseMove` (mode "bebas")

Tambahkan satu baris `AppendText` untuk `tebaltepi` sebelum baris penulisan warna dan garis:

```
Private Sub PictureBox1_MouseMove(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.MouseEventArgs) Handles PictureBox1.MouseMove
    Select Case modegambar
        Case "bebas"
            If dipencet Then
                Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                    g.DrawLine(tepi, titik, e.Location)
                End Using
                PictureBox1.Invalidate()

                ' === TAMBAHAN: simpan ukuran garis tepi ===
                TextBox1.AppendText("o tebaltepi " & tepi.Width.ToString & vbNewLine)

                TextBox1.AppendText("o warnatepi " & warnatepi.R.ToString & " " & _
                    warnatepi.G.ToString & " " & warnatepi.B.ToString & vbNewLine)
                TextBox1.AppendText("o garis " & titik.X.ToString & " " & _
                    titik.Y.ToString & " " & e.X.ToString & " " & e.Y.ToString & vbNewLine)

                titik = e.Location

                ' === TAMBAHAN: hitung baris yang ditambah (untuk Undo, soal 2) ===
                jmlBarisAksi += 3
            End If
        End Select
```

End Sub

b) Modifikasi pada event MouseUp

Sama seperti di atas, tambahkan baris tebal tepi sebelum baris-baris warna dan objek bangun:

```
Private Sub PictureBox1_MouseUp(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.MouseEventArgs) Handles PictureBox1.MouseUp
    Select Case modegambar
        Case "garis"
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawLine(tepi, titik, e.Location)
            End Using
        Case "kotak"
            Dim rect As New Rectangle(titik.X, titik.Y, e.X - titik.X, e.Y - titik.Y)
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawRectangle(tepi, rect)
            End Using
        Case "elips"
            Dim rect As New Rectangle(titik.X, titik.Y, e.X - titik.X, e.Y - titik.Y)
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawEllipse(tepi, rect)
            End Using
        Case "kotakisi"
            Dim rect As New Rectangle(titik.X, titik.Y, e.X - titik.X, e.Y - titik.Y)
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.FillRectangle(isian, rect)
            End Using
        Case "elipsisi"
            Dim rect As New Rectangle(titik.X, titik.Y, e.X - titik.X, e.Y - titik.Y)
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.FillEllipse(isian, rect)
            End Using
    End Select
    PictureBox1.Invalidate()
    dipencet = False

    If (titik.X <> e.X And titik.Y <> e.Y) Then
        ' === TAMBAHAN: simpan ukuran garis tepi ===
        TextBox1.AppendText("o tebaltepi " & tepi.Width.ToString & vbNewLine)

        TextBox1.AppendText("o warnatepi " & warnatepi.R.ToString & " " & _
            warnatepi.G.ToString & " " & warnatepi.B.ToString & vbNewLine)
        TextBox1.AppendText("o warnaisian " & warnaisian.R.ToString & " " & _
            warnaisian.G.ToString & " " & warnaisian.B.ToString & vbNewLine)
        TextBox1.AppendText("o " & modegambar & " " & titik.X.ToString & " " & _
            titik.Y.ToString & " " & e.X.ToString & " " & e.Y.ToString & vbNewLine)

        ' === TAMBAHAN: hitung baris yang ditambah (untuk Undo, soal 2) ===
        jmlBarisAksi += 4
    End If

    ' === TAMBAHAN: simpan ke stack Undo (soal 2) ===
    If jmlBarisAksi > 0 Then
        tumpukanUndo.Push(jmlBarisAksi)
    End If
End Sub
```

c) Modifikasi pada tombol "Gambar Ulang!" — membaca kembali tebal

Tambahkan satu Case baru bernama "tebaltepi" di dalam blok Select Case pecah(1). Saat baris ini dibaca, ubah tepi.Width dengan nilai yang tersimpan, sehingga penggambaran setelahnya akan memakai tebal yang sesuai aslinya.

```
Private Sub gbrUlang_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnGbrUlang.Click
```

```

btnBersihkan.PerformClick()
Dim a As Integer = TextBox1.Lines.Count
For i As Integer = 0 To a - 1
    Dim teksbaris As String = TextBox1.Lines(i)
    Dim pecah() As String
    pecah = teksbaris.Split(" ")
    On Error Resume Next
    pecah(1) = pecah(1).Trim(" ")
    Select Case pecah(1)

        ' === TAMBAHAN: kembalikan ukuran garis tepi saat Gambar Ulang ===
        Case "tebaltepi"
            tepi.Width = CSng(pecah(2))

        Case "warnatepi"
            warnatepi = Color.FromArgb(CByte(pecah(2)), CByte(pecah(3)), CByte(pecah(4)))
            tepi.Color = warnatepi
        Case "warnaisian"
            warnaisian = Color.FromArgb(CByte(pecah(2)), CByte(pecah(3)), CByte(pecah(4)))
            isian.Color = warnaisian
        Case "garis"
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawLine(tepi, CInt(pecah(2)), CInt(pecah(3)), _
                    CInt(pecah(4)), CInt(pecah(5)))
            End Using
        Case "kotak"
            Dim rect As New Rectangle(CInt(pecah(2)), CInt(pecah(3)), _
                CInt(pecah(4)) - CInt(pecah(2)), CInt(pecah(5)) - CInt(pecah(3)))
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawRectangle(tepi, rect)
            End Using
        Case "elips"
            Dim rect As New Rectangle(CInt(pecah(2)), CInt(pecah(3)), _
                CInt(pecah(4)) - CInt(pecah(2)), CInt(pecah(5)) - CInt(pecah(3)))
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.DrawEllipse(tepi, rect)
            End Using
        Case "kotakisi"
            Dim rect As New Rectangle(CInt(pecah(2)), CInt(pecah(3)), _
                CInt(pecah(4)) - CInt(pecah(2)), CInt(pecah(5)) - CInt(pecah(3)))
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.FillRectangle(isian, rect)
            End Using
        Case "elipsisi"
            Dim rect As New Rectangle(CInt(pecah(2)), CInt(pecah(3)), _
                CInt(pecah(4)) - CInt(pecah(2)), CInt(pecah(5)) - CInt(pecah(3)))
            Using g As Graphics = Graphics.FromImage(PictureBox1.Image)
                g.FillEllipse(isian, rect)
            End Using
    End Select
Next
PictureBox1.Invalidate()
End Sub

```

Contoh hasil baris vektor pada TextBox1 sesudah modifikasi

Sebagai ilustrasi, satu objek *Kotak Isi* berwarna biru dengan tebal tepi 5 akan menghasilkan empat baris di TextBox1:

```

o tebaltepi 5
o warnatepi 0 0 0
o warnaisian 0 0 255
o kotakisi 100 80 320 240

```

Karena baris tebaltepi selalu ditulis sebelum baris bangun, maka saat *Gambar Ulang!* dijalankan, urutan pembacaan akan: men-set tebal → men-set warna tepi → men-set warna isian → menggambar bangun. Inilah yang membuat hasil rekonstruksi sama persis dengan saat objek pertama kali dibuat.

Soal 2 — Membuat Tombol "Undo" Berfungsi

Soal: Buatlah agar tombol "Undo" berfungsi dengan cara mengurangi baris-baris terakhir data vektor yang dituliskan pada TextBox1 kemudian menggambar ulang seluruhnya. Jika misalnya terakhir kali ditambahkan 1 baris, hapus 1 baris. Jika 2 baris, hapus 2 baris.

Strategi Penyelesaian

Tiap satu kali aksi menggambar (satu siklus tekan–lepas mouse) dapat menambahkan jumlah baris yang berbeda-beda ke TextBox1, terutama mode *bebas* yang menambahkan banyak segmen garis selama mouse digerakkan. Karena itu, jumlah baris per aksi harus **dicatat** setiap kali aksi selesai, lalu disimpan dalam sebuah Stack(Of Integer). Saat tombol Undo ditekan, stack di-*pop* untuk mengetahui berapa baris yang harus dihapus dari bagian bawah TextBox1, kemudian kanvas digambar ulang dengan menekan btnGbrUlang.

Rincian jumlah baris yang ditambah per aksi (sudah memperhitungkan tambahan tebaltepi dari Soal 1):

Mode Gambar	Pemicu	Baris per Pemicu	Total per Aksi
bebas	MouseMove (per gerakan)	3 (tebaltepi, warnatepi, garis)	3 × N gerakan + 4 (saat MouseUp)
garis / kotak / elips / kotakisi / elipsisi	MouseUp (sekali)	4 (tebaltepi, warnatepi, warnaisian, bangun)	4

a) Tambahan deklarasi variabel global

Tambahkan dua variabel berikut di bagian deklarasi global (setelah Dim bmp As Bitmap):

```
' Variabel untuk fitur Undo
Dim tumpukanUndo As New Stack(Of Integer) ' menyimpan jumlah baris per aksi
Dim jmlBarisAksi As Integer = 0          ' penghitung baris untuk aksi yang sedang berjalan
```

b) Reset penghitung pada event MouseDown

Setiap aksi menggambar baru dimulai saat tombol mouse ditekan, sehingga penghitung harus di-nol-kan di sini.

```
Private Sub PictureBox1_MouseDown(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.MouseEventArgs) Handles PictureBox1.MouseDown
    tepi.Color = warnatepi
    isian.Color = warnaisian
    titik = e.Location
    dipencet = True

    ' === TAMBAHAN: reset penghitung baris untuk aksi baru ===
    jmlBarisAksi = 0
End Sub
```

Kode untuk MouseMove dan MouseUp sudah ditampilkan pada Soal 1 — kedua event itu yang melakukan `jmlBarisAksi += 3` (untuk mode bebas per gerakan) dan `jmlBarisAksi += 4` (saat mouse dilepas), lalu mendorong nilainya ke tumpukanUndo di akhir MouseUp.

c) Kode untuk tombol "Undo"

Tombol Undo melakukan empat hal: cek apakah masih ada aksi di stack → ambil jumlah baris terakhir → potong baris-baris itu dari TextBox1 → panggil btnGbrUlang.PerformClick() agar kanvas digambar

ulang dari sisa data vektor.

```
Private Sub btnUndo_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles btnUndo.Click
    ' Tidak ada aksi yang bisa di-undo
    If tumpukanUndo.Count = 0 Then
        Exit Sub
    End If

    ' Ambil jumlah baris yang ditambahkan oleh aksi terakhir
    Dim hapus As Integer = tumpukanUndo.Pop()

    ' Salin semua baris TextBox1 ke array baru, kecuali "hapus" baris terakhir
    Dim semua() As String = TextBox1.Lines
    Dim sisa As Integer = semua.Length - hapus
    If sisa < 0 Then sisa = 0

    If sisa = 0 Then
        TextBox1.Clear()
    Else
        Dim baruBaris(sisa - 1) As String
        Array.Copy(semua, baruBaris, sisa)
        TextBox1.Lines = baruBaris
    End If

    ' Gambar ulang seluruh kanvas dari data vektor yang tersisa
    btnGbrUlang.PerformClick()
End Sub
```

Penjelasan detail per baris

`tumpukanUndo.Pop()` mengembalikan sekaligus menghapus elemen teratas — yaitu jumlah baris dari aksi paling akhir.

`TextBox1.Lines` mengembalikan array string. Setiap kali kita memanggil setter `TextBox1.Lines = baruBaris`, isi `TextBox1` ditulis ulang persis sesuai array tersebut.

`Array.Copy(semua, baruBaris, sisa)` menyalin sisa baris pertama saja, sehingga hapus baris terakhir tertinggal dan otomatis terhapus.

`btnGbrUlang.PerformClick()` meng-emulasi klik tombol *Gambar Ulang!* sehingga kanvas dibersihkan lalu digambar ulang dari awal berdasarkan baris-baris yang masih tersisa.

Skenario uji singkat

1. Gambar sebuah **Kotak** → `MouseDown` menambah 4 baris, stack: [4].
2. Gambar coretan **Bebas** dengan 5 gerakan mouse → bertambah $(5 \times 3) + 4 = 19$ baris, stack: [4, 19].
3. Tekan **Undo** → pop 19 → hapus 19 baris terakhir → coretan bebas hilang, kotak tetap. Stack sekarang: [4].
4. Tekan **Undo** lagi → pop 4 → hapus 4 baris terakhir → kotak hilang, kanvas kosong. Stack: [].

Catatan Tambahan

Karena Soal 1 menambahkan baris baru pada `TextBox1` (yaitu tebal tepi), penghitung baris pada Soal 2 sudah menyesuaikan jumlahnya: 3 baris per gerakan mode *bebas* dan 4 baris untuk satu objek bangun pada `MouseDown`. Bila Soal 1 tidak diterapkan, angka tersebut harus dikurangi 1 (menjadi 2 dan 3).

Tipe data `tepi.Width` sebenarnya `Single`, sehingga saat membaca kembali pada *Gambar Ulang!* digunakan `CSng(pecah(2))` agar konsisten. Bila ingin lebih sederhana dan dijamin bilangan bulat (sesuai range `NumericUpDown` 1–10), `CInt(pecah(2))` juga dapat dipakai.